

Meerkat: Ranking Alert Families for Capacity-Limited Triage in Multi-Detector Security Operations

Jiacheng Wang

July 2026

Abstract

A Security Operations Centre runs detectors that turn raw telemetry into alerts. By the time an alert exists the detection has already happened, so the question left to the analyst is which alerts to open first. Seen from outside the role that sounds like a sorting problem. It is not.

A Tier 1 analyst works a shift with capacity for perhaps tens of investigations, while one monitored company in the dataset used here emits several thousand alerts on a day containing no attack at all. The imbalance is not merely one of volume. A single step of an intrusion can produce thousands of near-identical alerts, so the loudest activity on the network is not the most important, and a system that ranks by apparent urgency will spend the analyst’s entire day inside one event. Detectors disagree with each other: they use incompatible severity scales, describe the same machine under different names, and provide nothing that indicates whether two alerts belong to the same incident. Neither do they indicate when an alert is the only visible trace of a quiet step such as a privilege escalation, which produces a handful of records and matters more than the thousands beside it.

Published work approaches this problem from several directions. Adaptive Incident Prioritization and TEQ score items that a correlation layer has already assembled into incidents. AlertBERT learns which alerts belong to one attack. RAPID and DeepCASE rank or interpret alerts using surrounding context, and earlier work established analyst feedback and rarity as the core signals. Most of it assumes that the grouping problem has been solved upstream by a commercial product. An open-source deployment running Suricata, Wazuh and a log anomaly detector has no such layer, and must therefore decide for itself what a single unit of work is.

This report describes Meerkat, a triage pipeline built on that stack, and treats the choice of unit as the primary design variable rather than an implementation detail. Three successive designs are evaluated by leave-one-environment-out testing across the eight simulated companies of the AIT-ADS dataset, and each is measured by how much distinct attack activity is exposed within a fixed daily review budget, meaning the number of cases an analyst has time to open in a shift, rather than by per-alert accuracy. Equal space is given to the designs that failed, to a metric

that was withdrawn after it was found to measure redundancy while being read as quality, and to a set of later feature additions that improved conventional ranking scores while making the queue worse.

1 Introduction

1.1 Motivation

This work began as preparation for an internship. I wanted to join a Security Operations Centre and had never observed one from the inside, so my first concern was straightforward: I had no evidence that I could do the work.

Reading about the role supplied its vocabulary without supplying its difficulty. Accounts of Tier 1 triage agree on what the analyst does, which is to decide which alerts deserve investigation, and say comparatively little about which part of that decision is hard. I judged that the most reliable way to find out was to build the system myself: take the output of real detectors and produce the short list a human would work from.

The exercise became a research problem at the point where I measured it properly. My first design ranked individual alerts by predicted importance and performed well on the metrics I had been taught to report. It was useless in operation, and the reason had nothing to do with the model.

What the measurements showed is that the difficult part of triage lies before the model. It lies in deciding what counts as one thing to look at, and in deciding what counts as doing the job well. Under a fixed review capacity those two choices govern the outcome more than the choice of classifier does. That conclusion, rather than any particular model, is what this report argues.

1.2 Four observations that shaped the design

Alerts are not independent. One step of an attack can produce thousands of alerts. A single directory-scanning tool in this dataset generated 7,068 alerts against one host in about a minute. Treating those as 7,068 separate decisions is wrong in two ways: it consumes the whole review capacity, and it makes the loudest activity appear to be the most important.

Ranking alerts optimises the wrong objective. The dataset labels each step of its scripted attacks with an *attack window*, a time interval on a specific host during which that step took place. Windows differ enormously in how much noise they contain: four scanning windows hold 96.1% of all in-window alerts, while a privilege escalation window may hold only a handful of records. A model trained per alert weights each window by its alert count, so nearly the whole objective is supplied by the loudest few, and a quiet window can be missed at almost no cost to the training loss. Balancing the positive and negative classes does not help, because the imbalance at issue is among the positives.

A quiet window can still be learned if its alerts are distinctive enough to separate on their own. Section 3.5 reports a separate encoding defect in the first design that removed exactly that distinctiveness, so the two causes compounded.

Importance is not well defined until the metric is. An analyst with time for ten cases does not want the ten highest-scoring alerts if all ten describe the same scanning burst, so a per-alert precision score rewards a queue that repeats itself. This report instead measures how many *distinct* labelled events a day’s queue exposes, and several results that look strong under precision become weak under that measure.

That choice is a proxy and should be read as one. Counting distinct events treats every attack step as equally worth reaching, which is not what an analyst believes: a web shell uploaded to a public server and a port scan against a spare workstation are not the same finding. Measuring what genuinely matters would require impact information, meaning asset criticality and business consequence, which this dataset does not carry and which I have no operational experience to supply. Coverage is the strongest objective the available labels support, and Section 7 states what it leaves out.

The unit of work is the primary design variable. What the system scores matters more than how it scores. Most of the improvement reported here follows from changing what a single row represents rather than from changing the model above it.

1.3 Related work

Published triage systems mostly assume the ordering problem is posed over units that an earlier stage has already assembled. Microsoft’s Adaptive Incident Prioritization scores incidents that Defender has already correlated [4]. Sophos’ TEQ predicts actionability at alert and incident level using analyst escalations as labels [5]. AlertBERT learns to group alerts belonging to one attack [6]. RAPID ranks alerts using surrounding context [7], and DeepCASE analyses sequences of events for the same purpose [11]. Earlier work established analyst feedback and rarity as

core signals [9], and a recent survey maps the space of prioritization criteria and methods [8].

An open-source deployment running Suricata, Wazuh and a log anomaly detector has no correlation layer at all. It has to choose its own unit of work, and this report treats that choice as the primary design variable.

1.4 Contributions

The report is organised around five claims, each supported by a measurement reported later:

- A cross-detector normalization layer that derives a stable machine identity from a customer-supplied asset inventory, separating the machine an alert describes from the sensor that observed it.
- A measured negative result showing that per-alert ranking cannot reach the quiet phases of an intrusion under a fixed *review budget*, meaning the number of items an analyst can open in a day, whether ranked by a tuned model, by native severity or by name rarity.
- A two-level representation that removes the volume imbalance structurally rather than by reweighting: a *session*, one detector rule firing on one host until that host falls quiet, grouped into a *family*, one rule on one host across one day. The session boundary is chosen by nested validation rather than by intuition.
- A learned re-ranker over family context and a calibrated confidence that is shown to the analyst without being allowed to reorder the queue.
- A second negative result: three feature bundles that each improved the conventional ranking scores while losing window coverage, including one whose isolated discriminative power exceeded every feature already in the model.

2 Dataset

Experiments use AIT-ADS [1], published by the Austrian Institute of Technology under CC BY 4.0 and derived from the AIT log dataset [2]. It contains eight independently simulated enterprise environments, each monitored at the same time by Suricata (network intrusion detection), Wazuh (host intrusion detection) and AMiner (log anomaly detection). Each environment is subjected to a scripted multi-step attack made up of network scanning, service scanning, web enumeration (*dirb* and *wpscan*), web shell upload, credential cracking, reverse shell, privilege escalation, service disruption and DNS exfiltration.

The raw archive holds 2,655,821 alert records. Suricata alerts are collected by Wazuh and appear twice,

once decoded from `eve.json` and once re-parsed from `fast.log`. Removing the lower-information duplicates leaves 2,349,098 alerts across 79 labelled attack windows. The 306,723 removed records were checked and contain no event-labelled positive.

Two properties motivate the choice of dataset. Three concurrent detectors make cross-detector normalization a real rather than artificial problem, and eight independent environments allow training on seven while testing on one the model has never seen.

2.1 Label semantics

Each alert carries two labels produced by different processes, and the difference between them decides what can honestly be claimed. The field `time_label` names the attack phase active when the alert fired, derived from recorded phase intervals. The field `event_label` names the attack event the alert could be traced to in the underlying logs or network flows. Table 1 shows representative rows.

The second row is an antivirus update that happened to coincide with attacker scanning, so its phase label records timing and not malice. The fifth row sits inside the reverse shell window yet carries no event label, because the official labelling rules cover only selected log sources. The sixth row carries a `dnsteal` event label while its phase label is `false_positive`, which is real exfiltration outside the recorded interval.

Two aggregate properties govern everything that follows. First, event labels are heavily concentrated: `dirb` alone supplies 1,688,945 of 1,817,250 positives (92.9%). Second, and more importantly, the in-window alert population is concentrated the same way: four `dirb` windows hold 1,689,359 of the 1,758,559 in-window alerts, or 96.1%. Any objective defined over individual alerts therefore optimises mostly for one phase.

A window counts as **strictly reached** only when the queue holds an item containing an officially event-labelled alert from inside that window. Time overlap alone does not count. Of the 79 labelled windows only 60 contain such an alert, so 60 is a ceiling imposed by the supervision rather than by the method, and all coverage figures are read against it.

Finally, 72,212 event-labelled alerts fall outside every recorded window. The absence of a phase label therefore cannot be read as benign, and an absent event label means unlabelled rather than confirmed harmless.

2.2 What each detector produces

The differences between the three sources drive most of the normalization work and explain why a single severity ordering cannot be trusted.

Table 2 summarises the three sources. The volumes are strikingly uneven: Wazuh supplies 84.6% of

all alerts and AMiner 2.4%. The alert vocabulary is small in every case, between 24 and 34 distinct alert names, and in this corpus each name corresponds to exactly one rule identifier, so naming and rule identity carry the same information.

Detector	Alerts	Share	Names	Sev.
Wazuh	1,986,905	84.6%	24	3–10
Suricata	306,635	13.1%	29	1–3
AMiner	55,558	2.4%	34	none

Table 2: The three detectors across all eight environments. The severity column gives the range actually observed, not the range the scale permits.

The severity column is the first obstacle. Wazuh rule levels run from 0 to 15 with higher meaning more severe, and only levels 3 to 10 occur here. Suricata uses 1 to 3 with the ordering inverted, so 1 is the most urgent. AMiner emits no severity at all, because a log anomaly detector has no notion of rule gravity. Any cross-detector ordering therefore requires an explicit reconciliation, and the choice made for the detector that reports nothing is consequential: an early version assigned AMiner zero, which ranks an entire detector last by construction.

2.3 Which detector sees what

The three tools are complementary rather than redundant, and Table 3 shows how. Each cell is the share of that detector’s own alerts for which the field group carries a value.

Field group	Wazuh	Suricata	AMiner
Network endpoints	85.6%	100.0%	0.0%
Transport, app proto	85.4%	100.0%	0.0%
Flow volume	0.0%	100.0%	0.0%
TLS	0.0%	68.5%	0.0%
DNS	0.0%	30.1%	3.5%
HTTP	85.4%	1.3%	49.5%
Affected log paths	0.0%	0.0%	100.0%
Anomaly scoring	0.0%	0.0%	3.4%
Native ATT&CK	20.3%	0.0%	0.0%
Rule grouping	100.0%	100.0%	0.0%

Table 3: Field population by detector, as a share of that detector’s alerts.

Three consequences follow. First, no field is populated by all three detectors, so any feature built on a specific field is a feature about one detector unless absence is modelled explicitly. Second, the evidence an analyst wants is split across sources: Suricata alone reports flow volumes and TLS fingerprints, AMiner alone names the log file that triggered the anomaly, and only Wazuh supplies an ATT&CK tag, and then for a fifth of its alerts. Third, the display side of the tool cannot be organised by detector, because the same investigative question is answered by different sources depending on the alert.

Alert name	time_label	event_label
ClamAV database update	false_positive	–
ClamAV database update	network_scans	–
Multiple web server 400s	dirb	dirb
User changed UID	priv_esc	attacker_change_user
High entropy in request	reverse_shell	–
New ip address in DNS logs	false_positive	dnsteal

Table 1: Representative labelled alerts. The two labels are incomplete in different ways.

2.4 Where the positives are

The concentration by phase was given above. It is matched by a concentration across detectors: Wazuh supplies 93.4% of all labelled alerts, Suricata 4.0% and AMiner 2.5%.

A model trained on individual alerts therefore inherits two biases rather than one: it learns predominantly about a single attack phase, and predominantly from a single detector. Both disappear when the unit of work changes, which is the subject of Section 3.2.

Timestamps are also coarser than the data volume suggests. Wazuh reports whole seconds almost always, and 72.2% of alerts show a zero interval to the previous alert on the same machine. No feature depending on fine-grained ordering within a second could be built, which rules out sequence models of the kind DeepCASE applies [11].

2.5 A richer dataset that was deliberately not used

Microsoft’s GUIDE dataset is the obvious alternative to AIT-ADS and deserves an explicit account of why it was not used [10]. Published in 2024, it is drawn from real Defender telemetry and is larger than AIT-ADS by orders of magnitude, spanning millions of alerts and hundreds of thousands of incidents across thousands of organisations, mapped to hundreds of ATT&CK techniques. Its original labels are analyst verdicts recording whether each incident was escalated, judged benign, or dismissed as a false positive.

On 18 July 2026, by which point the design decisions below had been made and the evaluation was under way, it gained something closer still to the subject of this report: expert-derived prioritization labels for 9,980 incidents across 499 organisation queues, together with a protocol for evaluating how a system ranks a queue. That is a direct statement of the problem studied here, backed by real practitioner judgement rather than by the attack-window proxy of Section 7, and on a scale this dataset cannot approach.

By size, realism and label quality GUIDE is the better resource. Three things still make it the wrong instrument for this particular question.

The first is decisive and would remain so at any scale. GUIDE is Defender telemetry, and Defender

has already correlated raw evidence into incidents before anything is exported. The dataset therefore begins where the correlation layer ends, while the question here is what to do when no correlation layer exists at all. The central experiment of this report, comparing units of work against each other, cannot be run on data whose unit has already been decided upstream.

The second is that GUIDE is one vendor’s unified schema. Much of the work in Section 2.2 concerns three tools that disagree about severity scales, machine identity and which fields exist at all. That disagreement is the problem an open-source deployment actually has, and a single-vendor export does not contain it.

The third is timing, and it is a matter of record rather than oversight. The prioritization labels and the queue-ranking protocol were published on 18 July 2026, after this pipeline had been designed and while its evaluation was running. Adopting them would have meant restarting the evaluation on a different problem, and reporting a comparison assembled after the fact would have been worth less than reporting this one properly.

The correct conclusion is that GUIDE is the natural next test rather than a missed opportunity. The parts of this pipeline that depend on nothing AIT-specific, meaning the session and family construction and the coverage objective, could be evaluated against its ranking protocol directly, and the result would say something this report cannot: whether the choice of unit still matters when the labels come from analysts who were actually there.

3 Method

3.1 Normalization and asset identity

The three detectors emit incompatible JSON, with the field coverage reported in Section 2.2. Wazuh nests rule metadata under `rule`, Suricata events arrive wrapped inside a Wazuh envelope at `data.alert`, and AMiner uses an `AnalysisComponent` structure with epoch timestamps. Normalization maps all three onto one common table, which is the field mapping a SIEM performs when onboarding a log source.

The first decision is what the shared table should contain. Two options exist. The *intersection* keeps

only fields that every detector supplies, which guarantees no gaps and lets every feature be computed for every alert. The *union* keeps any field that any detector supplies, which preserves more evidence at the cost of leaving most columns empty for most rows.

Measured on this corpus the intersection is barely viable. Only twelve of the fifty-two columns are populated by all three detectors, and seven of those twelve are identifiers this pipeline constructs rather than anything a detector reports, such as the resolved machine identity and the position of the alert in its source file. What genuinely survives from all three tools is the timestamp, the alert name, the host string, the rule identifier and the requested web resource. A pipeline built on those five fields could group alerts and little else: it would have no severity to reconcile, no flow volumes, no process or user evidence, and no ATT&CK tag.

The union was therefore taken, and 43 of the 52 declared columns carry values for at least one detector. Wazuh populates 20 of them, Suricata 31 and AMiner 21, and 26 columns, three in every five, come from exactly one tool. That last figure is the argument for the choice: most of what the three detectors know is knowledge only one of them has, and an intersection discards all of it.

Two conventions make the sparsity safe to model. A missing string is the empty string and a missing number stays missing rather than becoming zero, so that a field a detector never reports is not read as a measured value of nothing. Features built on these columns are then either shared by construction, such as the reconciled severity of Section 2.2, or carry an explicit presence indicator. This is also why the analyst-facing evidence panels are organised by what a field means rather than by which detector produced it: a panel is shown when it has something to show, which follows from the union rather than from any per-detector rule.

Machine identity needs explicit treatment and was the source of the most damaging defect in this work. Wazuh omits its `predecoder` block for agent-collected files, and an early implementation fell back to `agent.name`, which is the constant string `wazuh-client`. That collapsed 1,698,994 of 1,986,905 Wazuh alerts (85.5%) onto one fictitious host, so contextual features counted activity across unrelated machines. Contextual counts on affected hosts were wrong by three orders of magnitude.

The pipeline now takes a customer-supplied asset inventory as an input, available before any alert is processed, and separates three identities:

- `entity_id`, the machine the alert describes, always an address the detector itself reported;
- `observer_id`, the sensor or collector that produced the alert;
- `entity_in_inventory`, whether the machine belongs to the company.

Suricata endpoints are resolved against the inventory with a destination-first rule. AMiner alerts are attributed using the log forwarding path, which corrects the cases where AMiner names the collector rather than the affected machine. An earlier discovery pass that inferred the inventory from the alert stream was removed: nine addresses across five environments had their only hostname evidence inside an attack window, so the inferred inventory depended on future information.

The official configuration decides membership and asset role. Display names come from the detector instead, because the two disagree. It disagrees with the identity Wazuh reports on 80,183 of 287,911 alerts (27.9%), in every case a permutation of external mail server names inside one environment. Group membership survives the permutation, so role-derived features stay valid while display names come from the detector.

3.2 From alerts to sessions to families

The unit scored by the ranker is the primary design variable of this work. The final pipeline groups in two stages.

A **session** is one detector rule firing on one host, closed once that host has been silent for longer than a gap parameter. A **family** collects a day’s sessions sharing one host, one detector and one rule:

(day, `entity_id`, `detector_source`, `rule_id`).

The family is the unit an analyst adjudicates. The decision “this rule fired 380,023 times on this host today” is one judgement, not 380,023 of them. The session exists underneath it because a family that spans a whole day mixes separate bursts of activity, and the analyst usually wants the burst.

Table 1 gives the reduction. The grouping also removes the volume imbalance structurally: each `dirb` flood becomes one row, so the 96.1% concentration reported above does not survive the change of unit. A family is positive when it holds at least one event-labelled alert, giving a positive rate near 11% that is stable across folds.

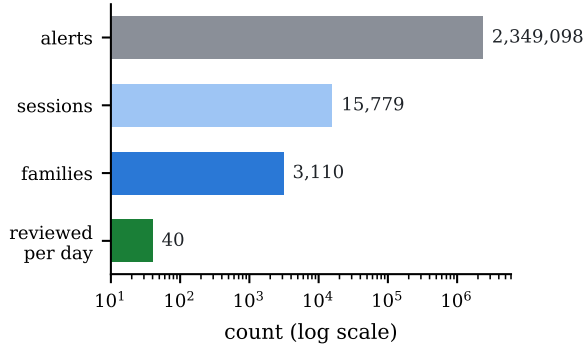


Figure 1: What grouping removes. Two and a third million alerts across the eight environments become 15,779 sessions and 3,110 families, of which an analyst reviewing ten a day opens forty over the four days of one environment. The horizontal scale is logarithmic.

3.3 Where the family unit fails

Collapsing thousands of alerts into one decision has an obvious risk. If a family holds seven thousand routine alerts and one genuinely malicious one, the analyst is shown a single summary of a mostly boring group, and the one alert that mattered can disappear inside it.

The reason this is rarer than it sounds is that the rule is part of the family key. Different steps of an intrusion generally trip different rules: the directory scanner fires a web error rule, the web shell upload fires a file upload rule, the privilege escalation fires a host rule. Each lands in a separate family and is scored on its own. Dilution can only happen when the malicious activity trips *the same* rule as the noise around it.

Measured on the demonstration environment, 47 families contain at least one alert the dataset traced to the attack, and in 42 of them the attack accounts for more than half the family. Dilution is the exception rather than the normal case. Two families are genuinely needles: one carries a single traced alert among 254, the other three among 178, both under the same Suricata rule. Neither reached the queue.

Three properties of the design push against the failure, and it is worth being precise about how far each goes. Sessions split a family wherever its activity pauses for ten minutes, so malicious activity separated in time from the noise becomes its own session with its own score. The best child score is one of the thirteen inputs to the re-ranker and carries its heaviest weight, so a single strong session raises the family it belongs to. Several aggregates are maxima rather than means, so one unusually severe alert propagates instead of being averaged away. And at review time the tool reports whether anything in the group succeeded, which surfaces a single successful request sitting among thousands of failures.

All three are defeated by the same case. An alert that is interleaved in time with the noise, carries no higher severity than it, and trips the same rule leaves

no trace in any aggregate the model reads, and only an analyst opening the underlying alert list would find it. The first of the two cases above is exactly that. This is a real cost of choosing the family as the unit of review, paid in exchange for the reduction reported in Figure 1, and it is not mitigated anywhere in the current pipeline.

3.4 Choosing the session gap

The gap is a real parameter and was chosen by measurement. Six schemes were compared on identical data, features, folds and scorer, with only the boundary rule changing: the plain daily family, and gap-closed sessions at 10, 60, 600, 3,600 and 21,600 seconds. The gap was selected by nested validation inside the training environments only, so the held-out environment never influenced the choice.

Table 4 reports the sweep. Ten seconds is degenerate: 72.2% of alerts show a zero interval to the previous alert on the same machine, because this data has no sub-second resolution, so the median unit becomes one alert and 53.5% are singletons. Ten minutes was the modal inner choice, winning in five of eight folds, and was frozen as a single constant because per-fold selection proved unstable across seeds while the constant did not.

Scheme	$K=5$	$K=10$	$K=25$
Daily family only	27	46	53
Gap 10 s	24	27	31
Gap 60 s	37	44	56
Gap 600 s	36	48	58
Gap 3,600 s	39	44	56
Gap 21,600 s	38	47	58

Table 4: Unit-boundary sweep. Windows reached of 79, single seed, 50 trees.

3.5 Features

Sessions are described by volume, duration, alert rate, reconciled severity, whether the detector supplied an ATT&CK technique, inventory membership, and context counted per machine per day. Families add aggregates of their child session scores together with span, alert count, cross-detector activity on the same host, and asset role.

Raw identities never become features. No IP address, hostname, rule identifier or alert name enters the matrix as a value, so a model trained on seven companies cannot recognise the eighth by name. The feature vocabulary is fitted on training environments only, so a rule first seen in the held-out company is treated as unseen rather than as familiar.

Two decisions correct defects of earlier designs. Rarity is a continuous count rather than a categorical vocabulary: the first design bucketed any alert name occurring fewer than fifty times into one shared cat-

egory, and those 38 names, carrying only 744 alerts, touch 33 of the 79 windows including all eight privilege escalation windows. The phases the first design never reached are largely the phases whose distinguishing names were removed before training.

Severity reconciliation maps three incompatible native scales onto $[0, 1]$. Wazuh reports 0–15 where higher is more severe, Suricata reports 1–3 where lower is more severe, and AMiner reports nothing and takes the midpoint. Assigning AMiner zero, as an earlier version did, ranks a whole detector last.

3.6 Scoring the session

Three models sit on this representation, one per level of the structure. A Random Forest scores each session, a logistic re-ranker scores each family, and a calibrator converts the family score into a figure an analyst can read as a probability.

The session model is a Random Forest with balanced class weights, since roughly one session in nine contains a labelled alert. It reads only the transferable description of Section 3.5: volume, duration, rate, reconciled severity, rarity, inventory membership and per-machine context.

3.7 The family re-ranker

A family usually holds several sessions, so its score has to be assembled from theirs. The obvious way is to take the highest child score and discard everything else. That choice throws away every property of the group: how many bursts there were, how long the activity ran in total, how many alerts it produced, and whether any other detector was active on the same machine at the same time.

The re-ranker exists to use those properties. It reads thirteen aggregates together with indicators for the asset’s inventory roles:

- the maximum, mean and spread of its child session scores, which describe both the strongest evidence and how consistent the sessions were;
- the number of child sessions, the total span in seconds and the total alert count, which separate a single tight burst from all-day drizzle;
- context counted per machine per day, and the number of distinct detectors active on that machine within ten minutes of the family, which is the corroboration signal. It is counted per machine rather than per family, since the family key already fixes the detector;
- the spread of observable taxonomy attached to the family, counted over Suricata alert categories, ATT&CK techniques supplied by the detector and rule groups.

The training boundary matters more than the model choice. The child scores that the re-ranker learns from must be produced by a forest that never saw those sessions, because a forest scoring its own training data returns an unrealistically clean signal, and a re-ranker fitted on it would learn a relationship that does not hold at inference. Child scores are therefore generated by an additional leave-one-environment-out pass inside the training environments, and the re-ranker is fitted on those out-of-fold scores.

A balanced logistic regression is used rather than a second forest. The family table holds roughly three thousand rows across eight environments, which is little data for a second ensemble, and a linear model over standardised inputs keeps each coefficient inspectable, so the contribution of any one aggregate can be read directly. Balancing is applied because about one family in nine is positive.

What this is worth against the simpler alternative is reported with the results.

3.8 Calibration and display

The re-ranker orders well and expresses confidence badly. On the held-out environment used for the worked examples it assigns scores above 0.999 to 25 of 326 families, and families scoring that high are genuine about 87% of the time. Presenting 0.999 to an analyst would invite a certainty the evidence does not support.

A one-variable logistic calibrator, fitted on out-of-fold re-ranker scores from the training environments only, maps the score onto the frequency actually observed at that score. The calibrated value is shown to the analyst and is never allowed to reorder the queue, which is ordered by the raw score alone, so a change to how confidence is displayed can never change whose case is opened first.

3.9 Enrichment and queue

Families are mapped to ATT&CK techniques and tactics [3] through a three-tier mapping: a reviewed override where one exists, otherwise the detector’s own tag, otherwise nothing. The mapping is rule-based so that every claim stays auditable. Only the vendor-supplied identifiers are used as a model feature; the hand-reviewed mapping was authored with knowledge of this dataset and is reserved for display, a decision Section 5.8 revisits with evidence.

The queue is the top K families per day, where K is a capacity setting a team chooses from the time it has.

4 Evaluation protocol

Evaluation is leave-one-environment-out: seven environments train, the eighth is held out, repeated eight times. The feature vocabulary, the forest, the re-ranker and the calibrator are all fitted without the held-out environment.

The primary metric is strict attack-window coverage at a fixed daily budget. It treats the queue as a set, so ten near-identical families do not score as well as ten families describing different activity. This is the practical Tier 1 question: with limited time, how many distinct suspicious activities did the queue expose?

What one unit of the budget costs. Because the designs compared here score different objects, K does not denote the same work in each. Under the first design it is K alerts; under the later two it is K families, and one family may contain thousands of alerts. The comparison is only meaningful if a family is genuinely a single decision.

The argument that it rests on what the analyst is being asked. A family is one detector rule firing on one machine during one day, so the question it poses is whether that rule firing on that machine today warrants investigation. That question is answered once, from a summary of the group, and the answer does not change with the number of alerts underneath it: a scanner that triggered a web rule seven thousand times is one judgement about one scanner, not seven thousand judgements. Supporting this in practice is a presentation requirement rather than a modelling one, and the accompanying tool is built for it, opening a family as a summary with its evidence grouped by meaning and the individual alerts available only on request.

This remains an assumption about analyst effort rather than a measurement of it, and the quantity it assumes away is not small. On the demonstration environment the forty families a ten-a-day budget selects over four days hold a median of 41 alerts each, a ninetieth percentile of 686 and a maximum of 7,068. Two in five hold fewer than ten alerts and would plainly be one decision; the largest is a scanner that fired for a minute, which is arguably also one decision but is not obviously the same amount of work as the first. Between those extremes the claim is untested.

No practitioner was timed reviewing either unit, and no user study was run, so the honest position is that this report measures how much attack activity a queue exposes and not what it costs to work through. If reviewing a large family does cost materially more than reviewing a small one, then the budgets are not strictly comparable across designs and the gain attributed to changing the unit is to that extent overstated. What can be said without assumption is the compression: those forty decisions span 15,653 of the

environment's 36,358 alerts, so 43 per cent of everything the detectors produced is represented in the queue. Section 7 returns to this.

The comparisons within the second and third designs are unaffected either way, since those score identical objects and differ only in how they are ordered.

Three protocol rules were adopted after an earlier iteration produced conclusions that did not survive scrutiny.

Budget curves, not a single budget. An earlier acceptance gate fixed at $K = 50$ had no derivation from analyst staffing, and conclusions drawn at it were an artefact of that choice.

An explicit saturation threshold. The family table holds roughly 78 items per day, so $K = 50$ reviews most of everything that exists and cannot separate rankings. Conclusions are restricted to $K \in \{5, 10, 25\}$, which review 6.4%, 12.9% and 32.2% of the table.

Every design is compared against untrained baselines. A learned score is only worth its complexity if it beats the orderings a team could write without any model. Four are used throughout: sorting by the severity the detectors already assign, by how rare the rule is, by how many alerts the item contains, and at random. The last is included because a metric that a shuffled queue scores well on is not measuring much. These baselines need no training, so they are applied directly to every environment.

Dominance, not significance. With eight folds sharing seven training environments each, paired tests are neither independent nor adequately powered. A method is accepted only if it is at least equal at every unsaturated budget and strictly better at one.

The forest size was swept at 50, 200, 300 and 500 trees and the re-ranker at three seeds. Two hundred trees is reported throughout, and Section 7 records how far the results move across that sweep.

Attack windows are known only during offline evaluation, and that separation is checked rather than assumed. Triaging one environment with the label files supplied, and again with them withheld, produces identical scores and an identical queue. The largest difference in any score is 2.2×10^{-16} , which is floating point noise. The only columns that change are evaluation columns no model reads.

5 Results

5.1 First design: ranking individual alerts

The first design encoded each alert as 35 numeric columns dominated by activity counts over 60, 600 and 3,600 second windows, and trained two Random Forests: a *context model* on attack-window membership and an *event model* on per-alert event labels, fused by maximum.

On conventional metrics it looks strong: 86.44% alert-level recall at 54.13% workload reduction and 98.87% average precision. Operationally it is not. Table 5 gives windows reached at 50 reviews per day.

Variant	Windows reached
Event model	26
Native detector severity	21
Full hybrid pipeline	20
Event and context fused	20
Context model	14

Table 5: First design. Windows reached at 50 reviews per day, summed over eight held-out environments, of 79.

Table 6 breaks that failure down by attack phase and shows where it falls. The design reaches the two web enumeration phases, which are the loud ones, and reaches nothing at all of the web shell upload, privilege escalation, credential cracking, reverse shell or network scanning. Those are the steps where an intruder is already inside, and they are the steps a triage system exists to surface.

Attack phase	Windows	Best reached
dirb	8	5
service_stop	8	4
wpscan	8	3
service_scans	8	1
dnsteal	8	1
webshell	8	0
priv_esc	8	0
cracking	7	0
reverse_shell	8	0
network_scans	8	0

Table 6: First design, coverage by attack phase at fifty alerts reviewed per day. Best reached is the highest any per-alert variant achieved for that phase, so the column is a per-phase maximum across variants and does not sum to the 20 windows any single variant reached. Five phases are reached by no variant at all.

The second row of Table 5 is the result that redirected the project. Sorting alerts by the severity the detectors already assign, a rule requiring no model, no training data and no features, reaches more attack windows than the complete learned pipeline built to replace it. Name rarity, an equally cheap heuristic, behaves comparably. A learned score that cannot

beat the number already printed on the alert is not earning its place, and no amount of tuning at the alert level changed that ordering.

Two further findings sit alongside it. Fusing the two scores by maximum costs six windows against the better component, because the dense context signal pushes event-ranked alerts out of a capacity-limited queue. And every `network_scans`, `privilege_escalation`, `reverse_shell`, `webshell` and `cracking` window is reached by no variant at all.

5.2 Diagnosis

Two measurements identify the cause. First, 96.1% of in-window alerts belong to four `dirb` windows, so an objective over alerts is overwhelmingly an objective about one phase, and the 86.44% recall above is largely a measurement of that phase. Second, about 93% of the context model’s importance falls on activity-volume counters, which is the representation the data invites.

The conclusion is that the scored unit, not the model family, is the binding constraint.

5.3 Second design: alert families

Changing the unit invites an obvious question: does the queue still need a learned model, or would something simpler do? Eight ways of ordering the same families were therefore run on identical data, folds and budgets, so that any difference between them comes from the score alone.

Four are heuristics that need no training and could be written in an afternoon. *Family size* puts the noisiest activity first. *Name rarity* puts the least common alert first, on the assumption that routine things are routine. *Reconciled native severity* trusts the priority the detectors already assign, once their three incompatible scales are mapped onto one. A *learned per-rule prior* scores each rule by how often it has historically been part of an attack, which tests whether the problem is really just knowing which rules matter.

One is unsupervised. An *Isolation Forest* scores how anomalous a family looks against all the others without ever seeing a label, which asks whether multivariate novelty over the whole feature set beats the single-variable rarity above. It appears twice, once as its own ranking and once as an extra feature handed to the supervised model, to test whether it finds anything the supervised model has not already extracted.

The two supervised orderings use a Random Forest. It was chosen for the shape of this problem rather than for accuracy: roughly three thousand families with around eleven per cent positive is little data, forests are robust at that size without tuning, they handle features on wildly different scales without rescaling, and their behaviour can be inspected feature by feature, which matters when the output is meant to justify itself to an analyst.

Table 7 reports the result.

Ordering	$K=5$	$K=10$	$K=25$
Random Forest	34	43	55
Random Forest + novelty	30	41	56
Isolation Forest novelty	24	30	44
Native severity	19	26	48
Family size	23	30	39
Name rarity	18	27	44
Per-rule prior	18	27	42
Share of table reviewed	6.4%	12.9%	32.2%

Table 7: *Second design. Attack windows reached, of the 79 the dataset labels, when the same alert families are put in different orders. K is the number of families reviewed per day. Novelty is an Isolation Forest score, which measures how unlike the others a family looks across all its features without using any label; it appears once as an ordering of its own and once as an extra input to the supervised model. The final row gives the share of all existing families that each budget reviews, which is why larger budgets stop discriminating.*

Budgets of fifty and one hundred were also measured and are omitted here. Past the saturation threshold of Section 4 every ordering reaches 59 or 60 windows, random order included, so the columns carry no information.

One choice behind these numbers decides what the model is taught to look for. A family counts as positive during training only when it holds an alert the dataset traced to the attack. Counting a family as positive merely for falling inside an attack window would add several hundred more examples, which is normally welcome when positives are scarce, and it was measured and was slightly worse. Time overlap is not evidence: the antivirus update of Section 2.2 runs while the attacker scans and is entirely innocent, so the looser label teaches the model that routine activity at an unlucky moment is worth surfacing. The stricter definition is used throughout.

The supervised ranker meets all three criteria fixed in advance. It beats family size, name rarity and the per-rule prior at every budget that can separate rankings, and it beats severity and the Isolation Forest as well. The learned score is therefore earning its place here, which it demonstrably was not at alert level.

Two smaller findings sit underneath that. The Isolation Forest reaches more windows than every hand-built heuristic, so looking at a family across all its features finds more than looking at any single one, even with no labels at all. Handing that same novelty score to the supervised model as an extra feature makes it slightly worse rather than better, which says the supervised model had already extracted whatever the unsupervised one found.

Gradient boosting was considered at this point and deliberately not run. CatBoost and XGBoost both often beat a Random Forest on tabular data, and the same argument rules out both, since what separates

them from a forest here is their handling of categorical fields. The only categoricals worth giving them are the rule identifier and the machine identity, and those are exactly the fields excluded from the feature matrix because they let a model memorise one company’s ruleset rather than learn anything transferable. That trick also needs data behind it. CatBoost replaces a categorical value with a running estimate of how often it has coincided with the target, so a rule identifier would become a number meaning “how often alerts from this rule turned out to be attacks”. With around three thousand alert families and three hundred positive ones, many of those estimates would rest on a handful of examples, and a model that trusts them is fitting noise rather than learning a pattern. Stripped of the categorical fields and run on the numeric ones alone, gradient boosting and a forest would probably finish close enough that an evaluation of this size could not tell them apart.

This is reasoning rather than measurement and should be read as such. Running one of them on the numeric features would have cost an afternoon and settled the question. The decision not to spend it rests on the result reported above: the same forest failed and then succeeded on identical data purely on the basis of what a row represented, so the evidence pointed at the representation rather than at the model. A stronger tabular model is nonetheless untested here, and a reader is entitled to observe that a tie was predicted rather than demonstrated.

5.4 Third design: sessions and a re-ranker

The third design keeps the family as the unit an analyst reviews and changes how that family is described and scored. It splits each family into the sessions of Section 3.2, so a rule firing in three separate bursts keeps that shape instead of being averaged into one flat description, and it replaces the rule for combining those sessions with the learned re-ranker of Section 3.7. Table 8 compares it against every alternative ordering of exactly the same families, under the same budget and the same counting rule, so the only thing that varies is the score.

Ranking method	$K=5$	$K=10$	$K=25$
Learned re-ranker	52	58	58
Best single child session	44	53	58
Family size (alert count)	29	30	39
Rule rarity, rarest first	23	32	47
Native detector severity	19	33	46
Random order	17	29	43

Table 8: *Third design. Windows strictly reached of the 60 that the labels allow, summed over the eight held-out environments. The two model rows are three-seed means at 200 trees; the four below them need no training and are scored directly, with random order averaged over three seeds.*

The rows below the model are the same alternatives as before, now applied to the two-level representation. *Best single child session* is the simplest way to score a family without a second model, taking its highest-scoring session and ignoring the rest. The three after it order by raw volume, by how unusual the rule is, and by the priority the detectors themselves assign. Random order is the floor, and is included because a metric that a shuffled queue scores well on would not be measuring much.

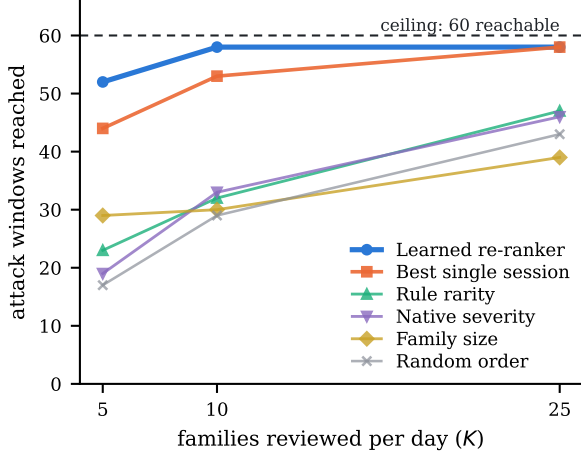


Figure 2: Attack windows reached as the daily review budget grows. The dashed line is the ceiling of 60 windows the labels permit. The learned re-ranker reaches it at ten families a day and gains nothing after that, while the untrained orderings are still climbing at twenty-five.

Figure 2 shows the same numbers as a budget curve. Two comparisons carry the result. The re-ranker against best single session isolates what reading the whole family is worth, since both see identical families and differ only in how the child sessions are combined: eight windows at a budget of five. The re-ranker against native severity, 52 against 19, is what the pipeline adds over simply trusting the number the detectors already print on the alert, and it is the comparison that failed at alert level.

Coverage also saturates at ten. Reviewing twenty-five families a day reaches no more windows than reviewing ten, which is worth knowing operationally: past the tenth case the extra work is not buying coverage, and the analyst’s time is better spent on the cases already surfaced.

Table 9 places the three designs side by side at equal review capacity and the same metric.

Design	$K=5$	$K=10$	$K=25$
Per-alert ranking	—	21	46
Alert families	34	43	55
Sessions + re-ranker	52	58	58

Table 9: The three designs at equal daily capacity, identical metric.

Read down that table, the per-alert row is the design that scored well on conventional metrics, and each later row costs nothing in analyst effort: every design reviews the same number of items a day. The improvement comes entirely from what those items are.

5.5 Coverage by attack phase

Table 10 breaks the final result down by scripted step.

Phase	Reach.	1st	$K=5$	$K=10$	$K=25$
wpscan	8	3	7	8	8
dirb	8	5	7	8	8
webshell	8	0	8	8	8
priv_esc	8	0	7	8	8
service_stop	8	4	7	7	7
service_scans	7	1	5	7	7
cracking	7	0	7	7	7
reverse_shell	4	0	4	4	4
network_scans	2	0	1	1	1
Total	60	—	53	58	58

Table 10: Windows reached per attack phase by the final design. Reach. is how many windows of that phase contain a labelled alert and can therefore be reached at all. 1st is the best any per-alert variant of the first design reached for that phase, at a budget five times larger; it is a per-phase maximum and so is not totalled; the remaining columns are the final design at each daily budget. Figures are the union over three seeds, so the $K=5$ column sums to 53 rather than the three-seed mean of 52.

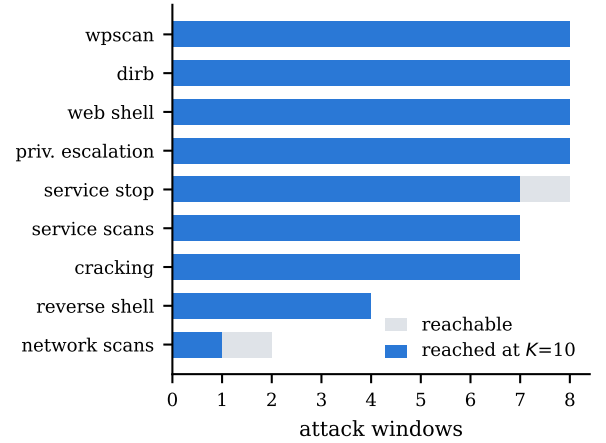


Figure 3: Coverage by attack phase at ten families a day. The pale bar is how many windows of that phase can be reached at all; the solid bar is how many were reached. Only network scanning falls materially short.

Figure 3 shows the same breakdown visually. In Table 10, the column headed *1st* carries the first design’s result for comparison: no per-alert variant reached a single web shell, privilege escalation, credential cracking or reverse shell window, and that was at five times the review capacity. Two windows stay

out of reach at any budget: one service stop and one network scan. Both are single-window steps on hosts the queue is already surfacing for other reasons, so the analyst is looking at the machine even though that particular window is not credited.

5.6 Calibration

Calibration quality is measured by the Brier score, which is the mean squared difference between a predicted probability and what actually happened, so a model claiming 90% confidence on cases that turn out genuine nine times in ten scores well and one claiming 99% on the same cases does not. Lower is better. It is used here rather than accuracy because the question is whether the displayed number can be believed, not whether the ordering is right.

The calibrator described in Section 3.8 lowers the pooled Brier score from 0.0301 to 0.0202 and improves every one of the 24 held-out environment and seed combinations.

The measured reliability curve locates the problem more precisely than the single Brier figure does. The raw score is close to honest at the extremes, where families scoring near zero are genuine 0.4% of the time and families scoring above 0.97 are genuine 94% of the time. It is the middle of the range that misleads: families scoring 0.84 are genuine 44% of the time and those scoring 0.95 are genuine 60%. An analyst reading those numbers as probabilities would substantially overestimate roughly one case in ten.

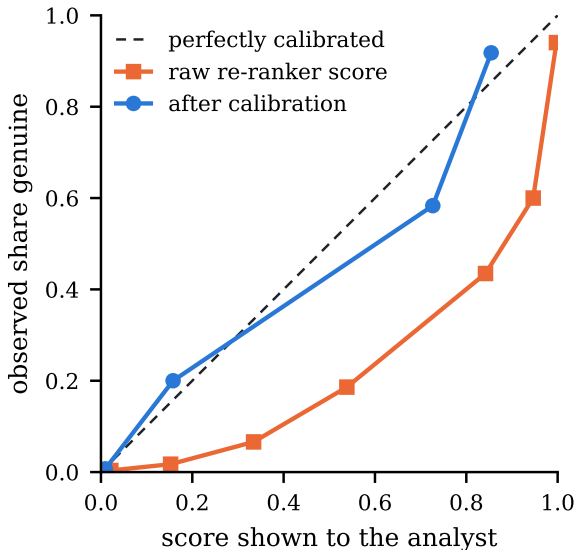


Figure 4: Reliability of the displayed score, measured over all 3,110 families in the eight held-out environments. A perfectly calibrated score lies on the diagonal, where a stated confidence matches the share of cases that turn out genuine. The raw re-ranker sits well below it through the middle of its range, claiming 0.84 for families genuine 44% of the time. Calibration pulls those points back towards the diagonal.

Figure 4 shows the effect it corrects. Its opera-

tional weight is small and should not be overstated. It changes no ordering, so no analyst opens a different case because of it, and the queue would be identical if the raw score were displayed instead. What it buys is honesty in the interface: the re-ranker emits values above 0.999 for families that turn out to be genuine about 87% of the time, and showing an analyst 0.999 invites a certainty the evidence does not support. It is a presentation component with a measurable justification, and nothing more.

5.7 What the model leans on

Both layers can be inspected, and Table 11 reports them. The forest figures are impurity importances grouped by what the features describe, averaged over the eight folds. The re-ranker figures are standardised logistic weights, so they are comparable with each other and carry a direction.

<i>Session forest, grouped importance</i>	
Asset role from the inventory	34.3%
Activity on that machine that day	26.7%
Rule rarity	18.1%
Family volume and rate	11.1%
Reconciled severity	5.8%
Detector identity	3.9%
Native ATT&CK presence	0.1%
<i>Family re-ranker, standardised weight</i>	
Best child session score	+1.86
Mean child session score	+0.89
Rule group count	-0.77
Asset role: dnat	+0.68
Asset role: servers	+0.62
Alerts on the machine that day	+0.59
Detectors nearby, ten minutes	+0.52
Asset role: firewall	+0.50
Families on the machine that day	-0.50
Spread of child scores	+0.39
Two further asset roles	+0.38

Table 11: What each layer relies on, measured over the eight held-out folds. Forest importances are grouped and sum to one hundred per cent. Re-ranker weights are standardised, so a positive value raises a family’s position and a negative one lowers it. The lower block lists every input with an absolute weight above 0.35, in order; the remaining features and roles fall below that and are omitted.

Three things are worth drawing out. Asset role is the single largest group in the session model, and it recurs in the re-ranker, where five separate roles carry weights between +0.38 and +0.68 and one of them ranks fourth of all inputs. Both layers therefore lean on information the inventory supplies and no detector reports, which is what makes the ablation below worth running. Native ATT&CK presence contributes essentially nothing at 0.1%, confirming that its value in this pipeline is analyst-facing context rather than ranking. And the re-ranker still leans

hardest on the best child session, which is the quantity the simpler baseline uses, with corroboration and asset role supplying the margin between them.

Asset role carrying the largest share raises a fair question. If the strongest signal in the model comes from a file the customer supplies, does the whole pipeline depend on being handed a good inventory? Removing every role feature from both layers and running the evaluation again answers it directly, and Table 12 reports the result.

Configuration	$K=5$	$K=10$	$K=25$
With the inventory	51	58	58
No asset-role feature	46	53	58
Best ordering needing no model	23	33	48

Table 12: Removing the inventory. Windows reached with every asset-role feature stripped from both layers, against the same configuration as shipped, at 50 trees and one seed. The final row is the strongest ordering that needs no model at all, included for scale.

The roles are worth five windows at the two budgets where analyst capacity binds, and nothing at the largest. That is a real contribution rather than a dependency. With no inventory information at all the pipeline still reaches 46, 53 and 58 windows, which is well ahead of every ordering that needs no model and ahead of the second design in Table 7. The system degrades when the inventory is missing instead of failing.

It is worth saying plainly that requiring an inventory is not an unusual demand. Any organisation large enough to run a Security Operations Centre already maintains a list of the machines it owns and what each one is for, usually in a configuration management database, and commercial triage products assume the same thing when they weight alerts by asset criticality. Meerkat asks for less than those products do, since it uses the inventory to identify the machine and its role and never to decide that one machine matters more than another. The requirement is realistic, and the ablation shows what is lost when it cannot be met.

The size of the contribution should still be read with the caution of Section 7. All eight environments come from one testbed and share a topology, so each has an intranet server, a mail server and a firewall, and the scripted attack targets comparable machines in each. A role such as `intranet` may therefore be acting partly as shorthand for “the kind of machine this script attacks”, which would carry between these eight environments while carrying poorly to a company whose attacker chose a different target. The two measurements also disagree in magnitude: 34.3% of the model’s decision weight buys five windows of sixty. That gap is itself informative, because an importance score reports what a model used, not what it could not have done without, and the ablation is the more trustworthy of the two.

Two weights are negative in a way worth noting. A family carrying many rule groups, and a machine carrying many separate families that day, both lower a family’s position. The plausible reading is that both mark ordinary busy machines rather than attacks, though impurity importances and linear weights are descriptive rather than causal, and establishing that would need an ablation this report does not run.

5.8 Feature bundles that failed

Once the pipeline above was frozen, the obvious next question was whether more information would help. Three additions were proposed, each with a reason to expect a gain:

- **Corroboration.** The model already counts how many detectors are active on a machine near the family in question. The bundle widened that idea: count distinct detectors and distinct rules over one minute and one hour rather than ten minutes alone. Several tools reacting to the same machine at the same time is the classic reason an analyst takes an alert seriously.
- **Curated ATT&CK.** The model reads only the technique tags the detectors supply themselves, and Section 2.2 shows Wazuh supplies one for a fifth of its alerts and the other two supply none. A reviewed mapping from rule identifiers to techniques covers far more, so feeding the model the richer mapping should sharpen it.
- **Severity and relative volume.** The family score never sees severity directly, only through its child sessions, and family size is absolute. Knowing that a rule fired forty times more than it usually does on that machine is more informative than knowing it fired forty times.

Each bundle was added to the frozen model on its own, so that a gain could be attributed to one change, and judged against a rule written down before any of them were run: coverage at a budget of five must improve by at least two windows, and coverage at ten and twenty-five must not fall. Table 13 reports the outcome.

Arm	$K=5$	$K=10$	Prec.	NDCG
Baseline	51	58	0.344	0.825
Corroboration	48	58	0.352	0.866
Curated ATT&CK	47	58	0.355	0.846
Severity + rel. volume	52	56	0.344	0.825

Table 13: Feature bundles against the frozen baseline, 50 trees, single seed.

None cleared the rule. More interesting than the failure is its direction: all three improved the two conventional ranking scores. Precision is the share

of queued families that were genuine, and NDCG rewards a ranking for placing genuine ones near the top rather than merely including them. Both rose in every case while window coverage stayed flat or fell. Judged the usual way, all three look like progress.

The curated ATT&CK bundle shows the mechanism most clearly. Replacing the detectors' sparse native tags with the reviewed mapping lifted the number of alerts carrying a technique from 403,026 to 1,747,323. The reconnaissance flood then carried a technique too, so the presence of an ATT&CK mapping stopped separating attack from noise and started pointing at the loudest activity on the network. This is also the evidence behind the decision in Section 3.9 to keep the reviewed mapping for display rather than for ranking.

A methodological note follows from this. Before fitting anything, each candidate feature was scored on its own for how well it separated genuine families from the rest. The best of them, counting distinct rules active near a family within the hour, separated them better than any feature already in the model. It then lost three windows when actually fitted, because the model already held several features that rise and fall with it, such as how busy the machine was that day. A feature measured alone tells you nothing about what it adds to a model that already has correlated information.

5.9 Where the queue still repeats itself

Grouping alerts into families removed one kind of repetition, where the same rule filled the queue many times over. A second kind survives it.

Much of what the model relies on describes how busy a machine was that day, so a machine under heavy attack produces many high-scoring families and can take every place in the queue. Counting how many distinct machines appear among the ten highest-scoring families each day, seven of the eight environments contain a day where all ten come from one machine, out of the eleven to seventeen machines that environment monitors.

This is partly correct behaviour, since the scripted attack does concentrate on one victim, and it still costs coverage. In one environment all ten selections belong to a single machine and reach fewer windows than the far simpler orderings by family size or severity, which spread across more machines and happen to catch activity elsewhere.

The cause is ranking itself rather than the model. Taking the top K by score solves

$$\max_{Q \subseteq \mathcal{F}, |Q|=K} \sum_{f \in Q} s(f),$$

where $\mathcal{F}$ is the day's families and $s(f)$ the learned score. Because the objective is a sum of independent terms, the best set is exactly the best K items taken

separately, and nothing in it depends on what the queue already holds. A tenth family that repeats the first nine scores as highly as one that shows something new.

Selecting the queue as a set replaces that with a coverage term. Let $A(f)$ be the observable attributes of a family, meaning its machine, its detector, its detector and rule pair, and the roles the inventory gives that machine. None of these requires investigating the family first. Writing $A(Q) = \bigcup_{g \in Q} A(g)$ for what the queue already covers, the marginal contribution of a candidate is the share of its attributes the queue does not yet hold,

$$\text{gain}(f \mid Q) = \frac{|A(f) \setminus A(Q)|}{|A(f)|} \in [0, 1],$$

and the queue is then built one family at a time, each step choosing

$$f^* = \arg \max_{f \in \mathcal{F} \setminus Q} \left[\alpha s(f) + (1 - \alpha) \text{gain}(f \mid Q) \right].$$

Normalising by $|A(f)|$ bounds the second term in $[0, 1]$ so that it cannot outweigh the score, and at $\alpha = 1$ the procedure reduces exactly to the ranked baseline, which is asserted in code. The frozen model is reused without retraining, so any difference comes from the selection step alone.

At α between 0.7 and 0.8 this recovers part of the loss, improving the weakest environment at every budget at no measurable cost in precision. Diversity alone is not a queue, however, and selecting purely for coverage reaches far fewer windows at precision 0.055 to 0.086.

6 Discussion

The result this project turns on is that the metric decided everything. Under precision or NDCG, three of the failed bundles look like improvements and the first per-alert design looks respectable at 86% recall. Under strict window coverage at a fixed budget, all four are correctly identified as worse. A team that measures triage by precision will keep making its queue narrower and more confident, and will keep missing the quiet half of an intrusion.

The second concerns infrastructure whose value depends on the formulation it serves. A stable machine identity across three detectors, and the asset inventory that supports it, were built to repair the attribution defect described in Section 3.1, at a point when the per-alert design had already been abandoned. Their contribution under that design was therefore never measured, and the comparison is chronological rather than controlled. What can be said is that the same work became substantial under family ranking, where asset role is the largest single group in the session model at 34.3% and the ablation above puts its worth at five windows.

The weaker half of that observation deserves stating too. Cross-detector corroboration was the original motivation for unifying machine identity, and it carries far less weight than expected, so the identity work paid off through asset context rather than through agreement between detectors.

Relative to prior work, this pipeline occupies the position that arises when the correlation layer assumed by AIP [4] and by incident-level TEQ [5] is absent. The family key is a fixed, explainable substitute for the learned grouping of AlertBERT [6], chosen because an analyst can audit it and it needs no training. The contribution is not the grouping technique, which is standard, but the measurement that the grouping is worth more than the ranker on this data, and that a widely used per-item precision metric can be actively misleading.

That last point is not hypothetical. A metric used earlier in this work, reported as “queue precision”, turned out to correlate -0.991 with the number of attack windows reached: it measured redundancy while being read as quality. It has been withdrawn from all conclusions.

7 Limitations

The results above are bounded by four separate things: what the labels can express, what the dataset represents, what the model was never given, and what I was able to bring to the problem. They are set out in that order because the first constrains everything after it.

7.1 The supervision is weak

Every number in this report is measured against labels that are incomplete by construction, and this is the most serious limitation.

The dataset supplies two labels per alert. One names the attack phase active when the alert fired, which records timing rather than causation: an antivirus update that happens during attacker scanning receives the scanning label while being entirely benign. The other names the attack event the alert could be traced to, and the official labelling rules only cover selected log sources, so an alert can sit inside an attack window, be caused by the attack, and still carry no event label. Section 2.2 gives a case of exactly that.

Three consequences follow. Nineteen of the 79 windows contain no event-labelled alert at all, so no queue can reach them under this definition and the effective ceiling is 60 rather than 79. An absent label means unlabelled rather than confirmed benign, so review rates outside windows are never reported here as false-positive rates, and any figure that reads like precision is precision against incomplete positives. And because the model is trained on these labels, it learns

what the labelling process captured rather than what the attack did.

A model trained on weak labels can only be as good as the labelling process, and no amount of modelling recovers a step the labels never recorded.

7.2 The data is simulated and unrepresentative in known ways

The eight environments come from one testbed running a shared attack script. They are consistent with each other in ways real companies would not be, so training on seven and testing on the eighth is a weaker test than it appears: the held-out environment is unseen but not unfamiliar. No claim about real-world performance follows from these results.

Attack prevalence ranges from 10.2% to 91.2% across environments, far above anything operational, where the great majority of days contain no intrusion at all. Timestamps are coarse, with 72.2% of alerts showing a zero interval to the previous alert on the same machine, which rules out any feature that depends on fine-grained ordering and closes off the sequence-based approach DeepCASE takes [11]. The attacks themselves are scripted and run during simulated working hours, so any feature keyed on time of day learns the testbed’s timetable; two such features were dropped before fitting for that reason.

7.3 What the model does not represent

The pipeline ranks by likelihood that a family is genuine, and carries no term for what would happen if it were. A critical public-facing server and a spare workstation showing identical activity receive identical treatment. This makes it a likelihood ranker and not a risk score, and it is the gap that separates the coverage metric from what an analyst actually optimises. Closing it needs asset criticality and business impact, which the dataset does not carry.

Several deliberate exclusions also bound what the model can do. Machine names, addresses, rule identifiers and alert names are kept out of the feature matrix so that a model trained on seven companies cannot recognise the eighth by name. This is necessary for the cross-environment claim and it discards information a real deployment would legitimately use, since a SOC does know which of its own machines matter. The reviewed ATT&CK mapping is likewise held back from the model because it was authored with knowledge of this dataset, and Section 5.8 shows that feeding it in makes the queue worse regardless.

7.4 The evaluation is weaker than a significance test

Dominance is reported rather than statistical significance. With eight folds that share seven training

environments each, paired tests are neither independent nor adequately powered, and none of the comparisons run clears a corrected threshold. The claims are therefore of the form “at least equal everywhere and better somewhere”, which is weaker than it looks in a table.

All eight environments informed design decisions at some point in the project, so none is a genuinely untouched test set. The protocol prevents a model from seeing its test environment during training; it does not prevent me from having seen it while choosing what to build.

Absolute coverage at the smallest budget is also unstable. Repeating the evaluation at 50, 200, 300 and 500 trees and across seeds moves coverage at a budget of five by several windows, and the three-seed closure reported here spans 51 to 53. Increasing the forest size does not monotonically help, and coverage at five was higher with 50 trees than with 500 in one sweep, which is not explained. The ordering between methods is stable across all of these configurations; the specific value at the tightest budget is not.

7.5 The author’s own limitations

The alternative dataset discussed in Section 2.5 would address several of the limitations above and was not used, for the reasons given there.

This work was done by a student with coursework in machine learning and security, and no operational experience in a Security Operations Centre. That shaped it in ways worth stating plainly, because they are not visible in any table.

Every judgement about what an analyst wants is inferred from published accounts rather than observed. The choice to make distinct-event coverage the objective, the decision that a rule firing on a machine in a day is the right unit to review, the layout of the evidence the tool displays, and the claim that ten cases a day is a plausible budget are all reasoned rather than validated. No analyst has used the tool, and no analyst reviewed its output. It is entirely possible that a practitioner would find the family the wrong unit, or the coverage objective a poor description of their working day, and nothing in this report would detect that.

The work was also unsupervised in the academic sense. A report of this kind would normally be produced under an advisor who challenges the framing, checks whether a result means what its author thinks, and knows from experience which directions are dead ends. I had documentation, published papers, large language models as a sounding board, and occasional email exchanges with teachers. Those substitute poorly for someone who can say that a measurement is wrong before six weeks are spent on it, and at least one such detour is recorded in this report. It is the main reason this document should be read as a project report and not as a peer-reviewed contri-

bution: it has had no adversarial review beyond my own, and nothing in it has been checked by anyone who does this work professionally.

The same inexperience narrowed which problems I recognised as problems. The first design’s failure was visible in the data from the start and I did not see it until I changed the measurement, because I was evaluating the model the way a machine learning course taught me to rather than the way the job demands. Several later decisions, including which candidate features were worth trying at all, were made on intuition about attacker behaviour that I have no field evidence for.

The inspection in Section 5.7 is descriptive for the same reason: impurity importances and linear weights report what a model used, not what caused the outcome, and no ablation was run to separate the two.

What the project can claim is that its measurements are honest and reproducible. What it cannot claim is that they measure the right thing for a real SOC.

8 Conclusion

Meerkat normalizes heterogeneous detector output against a customer-supplied asset inventory, groups it into sessions and daily families, ranks those families under a capacity-limited queue, enriches them with auditable ATT&CK context, and is evaluated across environments it has never seen.

Ranking individual alerts fails: 20 of 79 windows, none of the intrusion’s quiet phases, and worse than sorting on native severity. Ranking alert families succeeds: 34, 43 and 55 windows at 5, 10 and 25 reviews per day. Adding a session layer and a learned re-ranker reaches 52, 58 and 58 of the 60 label-reachable windows, against 19, 33 and 46 for the detectors’ own severity ordering, and calibration lowers the pooled Brier score from 0.0301 to 0.0202 across all 24 folds. Later attempts to extend the feature set did not improve coverage.

I started this project to find out what a SOC analyst’s problem actually is. The answer I did not expect is that most of it is decided before any model is trained, in the definition of what counts as one thing to look at and what counts as doing well.

References

- [1] M. Landauer, F. Skopik and M. Wurzenberger, *Introducing a New Alert Data Set for Multi-Step Attack Analysis*, Cyber Security Experimentation and Test Workshop (CSET), 2024.
- [2] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger and A. Rauber, *Maintainable Log Datasets for Evaluation of Intrusion De-*

- tection Systems*, IEEE Transactions on Dependable and Secure Computing, 2023.
- [3] MITRE, *MITRE ATT&CK Enterprise Matrix and STIX Distribution*, <https://attack.mitre.org>.
 - [4] S. Freitas, A. Gharib and M. Magenheimer, *Adaptive Incident Prioritization for Security Operations at Scale*, arXiv:2607.16963, 2026.
 - [5] B. Gelman, S. Taoufiq, T. Vörös and K. Berlin, *That Escalated Quickly: An ML Framework for Alert Prioritization*, arXiv:2302.06648, 2023.
 - [6] *AlertBERT: A Noise-Robust Alert Grouping Framework for Simultaneous Cyber Attacks*, arXiv:2602.06534, 2026.
 - [7] Y. Liu et al., *RAPID: Real-Time Alert Investigation with Context-aware Prioritization for Efficient Threat Discovery*, Annual Computer Security Applications Conference (ACSAC), 2022.
 - [8] *Alert Prioritisation in Security Operations Centres: A Systematic Survey on Criteria and Methods*, ACM Computing Surveys, 2024.
 - [9] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias and K. Li, *AP²: Training a Big Data Machine to Defend*, IEEE International Conference on Big Data Security, 2016.
 - [10] Microsoft, *GUIDE: a Dataset of Real-World Security Incident Triage Decisions*, 2024. Expert incident-prioritization labels for 9,980 incidents across 499 organisation queues, with a queue-ranking evaluation protocol, added 18 July 2026.
 - [11] T. van Ede et al., *DeepCASE: Semi-Supervised Contextual Analysis of Security Events*, IEEE Symposium on Security and Privacy, 2022.